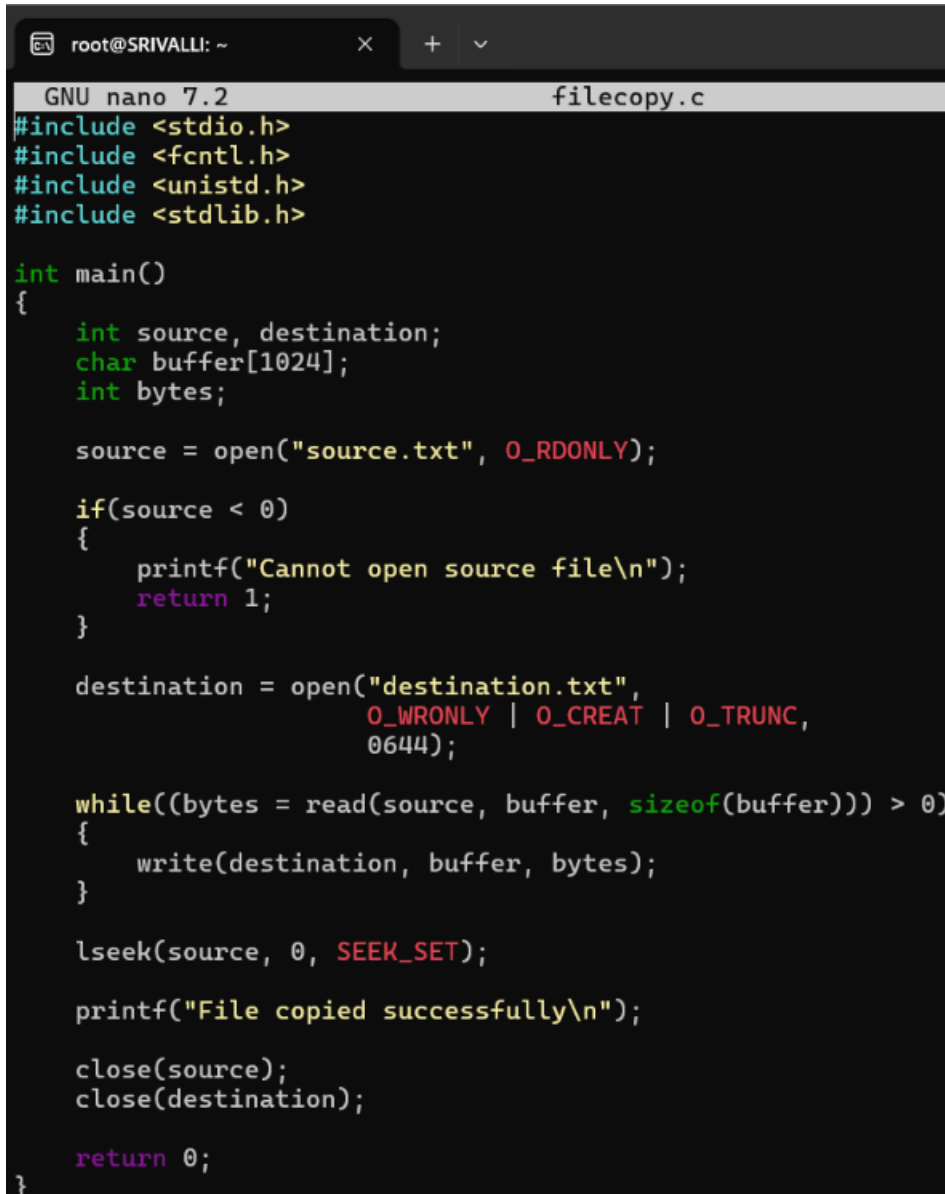


PRACTICAL_09

1. Implement a file copy utility using low-level Linux file I/O system calls (open, read, write, lseek, close). Compare its performance with standard library functions (fopen, fread, fwrite).

CODE:



```
GNU nano 7.2 filecopy.c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main()
{
    int source, destination;
    char buffer[1024];
    int bytes;

    source = open("source.txt", O_RDONLY);

    if(source < 0)
    {
        printf("Cannot open source file\n");
        return 1;
    }

    destination = open("destination.txt",
                      O_WRONLY | O_CREAT | O_TRUNC,
                      0644);

    while((bytes = read(source, buffer, sizeof(buffer))) > 0)
    {
        write(destination, buffer, bytes);
    }

    lseek(source, 0, SEEK_SET);

    printf("File copied successfully\n");

    close(source);
    close(destination);

    return 0;
}
```

```

GNU nano 7.2                                     filecopy_std.c
#include <stdio.h>

int main()
{
    FILE *src, *dest;
    char buffer[1024];
    size_t bytes;

    src = fopen("source.txt", "r");
    dest = fopen("destination2.txt", "w");

    while((bytes = fread(buffer, 1, sizeof(buffer), src)) > 0)
    {
        fwrite(buffer, 1, bytes, dest);
    }

    fclose(src);
    fclose(dest);

    printf("File copied using standard library\n");

    return 0;
}

```

RESULT:

```

root@SRIVALLI:~# nano filecopy.c
root@SRIVALLI:~# nano source.txt
root@SRIVALLI:~# gcc filecopy.c -o filecopy
root@SRIVALLI:~# ./filecopy
File copied successfully
root@SRIVALLI:~# cat destination.txt
root@SRIVALLI:~# nano filecopy_std.c
root@SRIVALLI:~# gcc filecopy_std.c -o filecopy_std
root@SRIVALLI:~# ./filecopy_std
File copied using standard library

```

REPORT:

In this experiment, a file copy utility was developed using low-level Linux system calls such as `open()`, `read()`, `write()`, and `close()`. The copied output was verified and compared with a file copy program using standard library functions like `fopen()`, `fread()`, and `fwrite()`. This experiment helps in understanding file handling and file I/O operations in Linux.

2. Develop a program that redirects standard input and Output using file descriptors and `dup2()`. Demonstrate how shells implement VO redirection.

CODE:

```
root@SRIVALLI: ~
GNU nano 7.2 io_redirection.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main()
{
    int input_fd, output_fd;
    char data[100];

    input_fd = open("input.txt", O_RDONLY);

    output_fd = open("output.txt",
                    O_WRONLY | O_CREAT | O_TRUNC,
                    0644);

    dup2(input_fd, STDIN_FILENO);
    dup2(output_fd, STDOUT_FILENO);

    scanf("%s", data);

    printf("Data Read: %s\n", data);

    close(input_fd);
    close(output_fd);

    return 0;
}
```

```
root@SRIVALLI: ~
GNU nano 7.2
HelloLinux
```

RESULT:

```
root@SRIVALLI:~# nano io_redirection.c
root@SRIVALLI:~# nano input.txt
root@SRIVALLI:~# gcc io_redirection.c -o io_redirection
root@SRIVALLI:~# ./io_redirection
root@SRIVALLI:~# cat output.txt
Data Read: HelloLinux
```

REPORT:

In this experiment, standard input and standard output were redirected using file descriptors and the `dup2()` system call. Input was read from a file and output was written to another file instead of the terminal. This demonstrates how Linux shells implement input/output redirection using file descriptor manipulation and stream redirection